

HRMS Payroll & Leave Management System

Technical Documentation, Setup Guide & API Reference

Spring Boot + React + MySQL | JWT Security | PDF Payslips | Email Automation

1. Project Overview

The HRMS Payroll & Leave Management System is a full-stack application for employee management, salary structure configuration, employee salary assignment, attendance, leave management, payroll processing, payslip PDF generation and email delivery. The backend uses Java/Spring Boot, Spring Data JPA/Hibernate, Spring Security/JWT and MySQL. The frontend uses React, Vite and Axios.

2. Objectives

- Manage employee records and roles.
- Configure salary structures containing Basic Salary, HRA and Special Allowance.
- Assign salary structures to employees and retain effective dates/history.
- Manage CL, SL and EL leave balances.
- Apply, approve and reject employee leave requests.
- Automatically deduct approved leave from the applicable balance.
- Generate monthly payroll and PDF payslips.
- Email payslips to employees and maintain email delivery logs.
- Support scheduled payroll processing.

3. Technology Stack

Layer	Technology
Backend	Java 17+, Spring Boot, Spring Web, Spring Data JPA, Hibernate
Security	Spring Security, JWT Authentication, Role-Based Access Control
Database	MySQL
Frontend	React.js, Vite, JavaScript, Axios, HTML5, CSS3
Build	Maven, npm
Email	SMTP / JavaMail
Documents	PDF payslip generation
Tools	Postman, MySQL Workbench, Git, GitHub

4. High-Level Architecture

React frontend → Axios → Spring Boot REST APIs → Service Layer → Spring Data JPA/Hibernate → MySQL. Payroll processing also invokes PDF generation and email services. JWT is used for authenticated requests.

Main backend packages: controller, service, repository, entity, security and configuration.

5. Functional Modules

Employee Management: Create, read, update and delete employees; employee code, name, email, department, designation and role.

Authentication: Email/password login, JWT authentication and protected APIs.

Salary Structure: Create and manage Basic Salary, HRA, Special Allowance and effective dates.

Employee Salary Assignment: Assign a salary structure to an employee with an effective-from date.

Leave Management: Apply leave, calculate leave days, approve/reject requests and maintain leave history.

Leave Balance: Display total, used and remaining CL/SL/EL balances.

Attendance: Maintain attendance information used by payroll processing.

Payroll: Generate monthly payroll, calculate earnings/deductions/net salary and store payroll records.

Payslip: Generate, store, view and regenerate PDF payslips.

Email Automation: Send leave status notifications and payslips through SMTP; maintain delivery logs.

Scheduled Payroll: Provide scheduled/manual payroll processing for eligible employees.

6. Business Flow

Employee → Salary Structure Assignment → Attendance/Approved Leave → Payroll Calculation → Payroll Record → Payslip PDF → Email Delivery

Leave flow: Employee applies → PENDING → HR/Admin approves or rejects. Approval checks the balance, deducts the required days, changes status to APPROVED and sends an email. Rejection changes status to REJECTED without deducting the leave balance and sends a rejection email.

7. Leave Policy

Leave Type	Code	Annual Allocation
Casual Leave	CL	12
Sick Leave	SL	12
Earned Leave	EL	15

8. Setup Instructions

8.1 Prerequisites

- Java 17 or newer
- Maven
- Node.js and npm
- MySQL Server
- Git
- Postman (recommended for API testing)

8.2 Database Setup

Create the database in MySQL:

```
CREATE DATABASE hrms;
```

Update the backend application.properties with the MySQL URL, username and password. The project documentation uses jdbc:mysql://localhost:3306/hrms and Hibernate ddl-auto=update.

Example properties:

```
spring.datasource.url=jdbc:mysql://localhost:3306/hrms
```

```
spring.datasource.username=root
```

```
spring.datasource.password=YOUR_MYSQL_PASSWORD
```

```
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true
```

```
spring.jpa.properties.hibernate.format_sql=true
```

8.3 Email/SMTP Setup

Configure SMTP in application.properties. For Gmail, use a Google App Password instead of the normal Gmail password. Do not commit SMTP credentials to GitHub.

```
spring.mail.host=smtp.gmail.com
```

```
spring.mail.port=587
```

```
spring.mail.username=YOUR_EMAIL@gmail.com
```

```
spring.mail.password=YOUR_APP_PASSWORD
```

```
spring.mail.properties.mail.smtp.auth=true
```

```
spring.mail.properties.mail.smtp.starttls.enable=true
```

For production, supply credentials through environment variables such as MAIL_USERNAME and MAIL_PASSWORD.

8.4 JWT Configuration

Keep the JWT secret outside source control. Example:

```
jwt.secret=${JWT_SECRET}
```

8.5 Start Backend

From the backend project directory:

```
mvn spring-boot:run
```

Windows wrapper:

```
mvnw.cmd spring-boot:run
```

Default backend URL: <http://localhost:8080>

8.6 Start Frontend

From the frontend directory:

```
npm install
```

```
npm run dev
```

Default frontend URL: <http://localhost:5173>

Frontend API base URL: <http://localhost:8080/api>

9. Recommended Testing Sequence

1. Start MySQL and create the hrms database.
2. Configure database, SMTP and JWT settings.
3. Start the Spring Boot backend.
4. Start the React frontend.
5. Login and obtain the JWT.
6. Create employee records.
7. Create salary structures.
8. Assign salary structures to employees.
9. Configure yearly leave balances.
10. Create attendance records.
11. Apply leave and test approval/rejection.
12. Verify leave balance deduction and email notification.
13. Generate monthly payroll.
14. View/regenerate the payslip PDF.
15. Send the payslip by email and verify the email log.

10. API Documentation

Base URL: <http://localhost:8080>

Authenticated endpoints should be called with the JWT returned by the login API, according to the application's Spring Security configuration.

Module	Method	Endpoint	Purpose
Authentication	POST	/api/auth/login	Login and obtain authentication information
Employee	GET	/api/employees	Get all employees
Employee	GET	/api/employees/{employeeId}	Get employee by ID
Employee	POST	/api/employees	Create employee
Employee	PUT	/api/employees/{employeeId}	Update employee
Employee	DELETE	/api/employees/{employeeId}	Delete employee
Salary Structure	GET	/api/salary-structures	Get salary structures
Salary Structure	GET	/api/salary-structures/{id}	Get salary structure by ID
Salary Structure	POST	/api/salary-structures	Create salary structure
Salary Structure	PUT	/api/salary-structures/{id}	Update salary structure
Salary Structure	DELETE	/api/salary-structures/{id}	Delete salary structure
Employee Salary	GET	/api/employee-salary	Get all salary assignments
Employee Salary	GET	/api/employee-salary/{id}	Get salary assignment by ID
Employee Salary	POST	/api/employee-salary/{employeeId}/{salaryStructureId}?effectiveFrom=YYYY-MM-DD	Assign salary structure
Leave	POST	/api/leaves/apply/{employeeId}	Apply for leave
Leave	GET	/api/leaves	Get all leaves
Leave	GET	/api/leaves/employee/{employeeId}	Get employee leave history
Leave	PUT	/api/leaves/{leaveId}/approve	Approve leave
Leave	PUT	/api/leaves/{leaveId}/reject	Reject leave
Leave Balance	GET	/api/leaves/employee/{employeeId}/balance	Get leave balance
Attendance	GET	/api/attendance	Get attendance records
Attendance	GET	/api/attendance/{id}	Get attendance by ID
Attendance	POST	/api/attendance	Create attendance
Attendance	PUT	/api/attendance/{id}	Update attendance
Attendance	DELETE	/api/attendance/{id}	Delete attendance
Payroll	GET	/api/payroll	Get all payrolls
Payroll	GET	/api/payroll/{payrollId}	Get payroll by ID
Payroll	POST	/api/payroll/generate/{employeeId}?month=MM&year=YYYY	Generate payroll
Payroll	GET	/api/payroll/employee/{employeeId}	Get employee payroll history
Payroll	GET	/api/payroll/employee/{employeeId}/period?month=MM&year=YYYY	Get payroll for period
Payslip	GET	/api/payroll/{payrollId}/payslip/view	View generated PDF
Payslip	POST	/api/payroll/{payrollId}/payslip/regenerate	Regenerate PDF
Payslip Email	POST	/api/payroll/{payrollId}/payslip/email	Email payslip
Payroll	DELETE	/api/payroll/{payrollId}	Delete payroll
Scheduled Payroll	POST	/api/payroll/schedule/run	Run scheduled payroll process

11. Important API Examples

11.1 Login

POST /api/auth/login

```
{"email":"rahul@example.com","password":"123456"}
```

Important: the field name is password, not pasword.

11.2 Apply Leave

POST /api/leaves/apply/4

```
{"leaveType":"CL","startDate":"2026-08-15","endDate":"2026-08-16","reason":"Personal work"}
```

11.3 Get Leave Balance

GET /api/leaves/employee/4/balance

```
{"employeeId":4,"employeeName":"Rahul","year":2026,"casualLeaveTotal":12,"casualLeaveUsed":2,"casualLeaveRemaining":10,"sickLeaveTotal":10,"sickLeaveUsed":1,"sickLeaveRemaining":9,"earnedLeaveTotal":15,"earnedLeaveUsed":3,"earnedLeaveRemaining":12}
```

11.4 Generate Payroll

POST /api/payroll/generate/4?month=8&year=2026

The process uses employee salary assignment, attendance and approved leave information, creates the payroll record and generates a payslip PDF.

11.5 View Payslip

GET /api/payroll/5/payslip/view

The endpoint returns the generated PDF as application/pdf for browser viewing.

11.6 Send Payslip

POST /api/payroll/5/payslip/email

The system sends the payslip to the employee's registered email and stores delivery information.

12. Postman Collection

A starter Postman collection is included with this documentation as a separate JSON file. Import the collection into Postman, set the baseUrl variable to http://localhost:8080, execute Login first, then use the token for protected requests.

Suggested Postman environment variables:

- baseUrl = http://localhost:8080
- employeeId = 4
- salaryStructureId = 1

- payrollId = 5
- leaveId = 12
- token = <JWT returned by login>

13. Screenshot / Evidence Section

The following four screenshots are required for the final submission. The current project evidence contains three uploaded screenshots, which are included below where relevant. The four requested UI screenshots should be captured from the final working frontend and inserted in the marked locations.

13.1 Salary Structure Screen

Salary Structures

Define and manage salary structures

Add Salary Structure

Name

e.g. Software Engineer - L2

Basic Salary

0.00

HRA

0.00

Special Allowance

0.00

Effective From

dd-mm-yyyy

Effective To (optional)

dd-mm-yyyy

Add Structure

All Salary Structures

ID	NAME	BASIC	HRA	SPECIAL ALLOWANCE	GROSS	EFFECTIVE FROM	EFFECTIVE TO	STATUS	ACTIONS
1	Rahul Sharma	₹ 20,000.00	₹ 10,000.00	₹ 5,000.00	₹ 35,000.00	2026-01-01	-	Active	Edit Dele
2	SDE-2	₹ 5,00,000.00	₹ 10,000.00	₹ 15,000.00	₹ 5,25,000.00	2026-08-12	2026-08-27	Inactive	Edit Dele

13.2 Leave Balance Screen

Leave Application

Apply for employee leave

Employee

EMP001 - Rahul Sharma

Employee

Rahul Sharma

Department

Dev

Designation

Java Developer

Email

rahul@example.com

Leave Balance

Available leave for selected employee

Refresh

Casual Leave

9

days remaining

Total: 9

Used: 3

Sick Leave

10

days remaining

Total: 10

Used: 2

Earned Leave

13

days remaining

Total: 13

Used: 2

Leave Type

Casual Leave (CL)

Number of Days

0

13.3 Generated Payslip

EMPLOYEE PAYSリップ

Employee Code: EMP002

Employee Name: Souvik Mukerjee

Department: CSE-DS

Designation: Student

Pay Period: 10/2027

Salary Component	Amount
Basic Salary	500000.00
HRA	10000.00
Special Allowance	15000.00
Gross Salary	525000.00
LOP / Attendance Deduction	0.00
PF	60000.00
ESI	0
Professional Tax	200.00
Total Deductions	60200.00
NET SALARY	464800.00

This is a system generated payslip.

13.4 Email Sent Confirmation

Payslip Email Logs

Track payslip email delivery history

Refresh

Employee

Select employee

Search Employee

Show All

Total Emails

4

Sent

4

Failed

0

ID	EMAIL	STATUS	SENT AT	ERROR MESSAGE
3	rahul@example.com	SENT	13 Aug 2026, 03:42 pm	-
4	vedantverma147@gmail.com	SENT	13 Aug 2026, 06:27 pm	-
5	rahul@example.com	SENT	14 Aug 2026, 02:22 am	-
6	rahul@example.com	SENT	14 Aug 2026, 11:34 am	-

13.5 Available Development Evidence

POST http://localhost:8080/api/auth/login Send

rams Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

```
1 {
2   "email": "rahul@example.com",
3   "password": "123456"
4 }
```

ly Cookies (1) Headers (14) Test Results Status: 401 Unauthorized Time: 7 ms Size: 458 B Save Response

Key	Value
Vary	Origin
Vary	Access-Control-Request-Method
Vary	Access-Control-Request-Headers
X-Content-Type-Options	nosniff
X-XSS-Protection	0
Cache-Control	no-cache, no-store, max-age=0, must-revalidate
Pragma	no-cache

Frontend API Authorization Error During Development

<https://react.dev/link/react-devtools>

✖ Failed to load resource: the server responded with a status of 403 ()

:8080/api/employees:1

✖ ▶ AxiosError: Request failed with status code 403

Employee.jsx:62

Show ignore-listed frames

✖ Failed to load resource: the server responded with a status of 403 ()

:8080/api/employees:1

✖ ▶ AxiosError: Request failed with status code 403

Employee.jsx:62

Show ignore-listed frames

Login response: ▶ Object

Login .jsx:37

MySQL Employee Salary Assignment Data

```
mysql> use hrms;
Database changed
mysql> select * from employee_salary;
+-----+-----+-----+-----+-----+
| id | effective_from | effective_to | employee_id | salary_structure_id |
+-----+-----+-----+-----+-----+
| 2 | 2026-08-12 | NULL | 1 | 1 |
| 3 | 2026-01-01 | NULL | 1 | 1 |
| 4 | 2026-08-04 | NULL | 3 | 1 |
| 5 | 2026-07-29 | NULL | 1 | 1 |
| 6 | 2026-08-12 | NULL | 3 | 2 |
+-----+-----+-----+-----+-----+
5 rows in set (0.01 sec)

mysql>
```

14. Payslip Storage

Generated payslips are stored under the payslips/ directory. The generated file path is stored in the payroll record.

Example: payslips/Payslip_EMP001_8_2026.pdf

15. Security & Configuration Notes

- Do not commit database passwords, SMTP passwords/app passwords or JWT secrets to GitHub.
- Use environment variables for production credentials.
- Passwords should be stored using secure hashing rather than plain text in production.
- Protected APIs should be accessed using the JWT returned after login.
- Use HTTPS in production.
- Restrict CORS origins to trusted frontend domains in production.

16. Common Issues & Troubleshooting

Login returns 401: Verify email/password, confirm the request property is password, and verify the employee exists with the expected credentials.

Employee API returns 403: Check the JWT token, Spring Security authorization rules and whether the logged-in role is allowed to access the endpoint.

Leave balance returns 500: Verify a leave_balances row exists for the employee and the current year.

Employee salary assignment not found: Check the requested employee_salary ID. GET /api/employee-salary returns the existing assignment IDs.

Payslip cannot be viewed: Verify that the payroll record has a pdfPath and that the file exists in the payslips directory.

Email is not sent: Verify SMTP host/port, username, app password, STARTTLS settings and network access.

Frontend cannot reach backend: Confirm backend is running on port 8080 and frontend Axios uses `http://localhost:8080/api`.

17. Conclusion

The HRMS module provides an end-to-end payroll and leave workflow: employee and salary configuration, leave tracking and approval, attendance integration, monthly payroll generation, PDF payslip creation and automated email delivery. The documented API and setup sequence provide a reproducible path for running and testing the system.